SOC Onboarding — Microsoft Sentinel + Defender XDR Unified Incidents

Implementation Solution Guide

Version 1.0.0 · 2026-07-09

Contents

Executive Summary — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Boundary . . . 4

Outcome . . . 5

1. Solution Overview — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 5

What this solution is . . . 5

How the connection works . . . 5

The objects this solution creates (named to convention) . . . 6

What you can control . . . 6

Why this matters (the business case) . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 7

2.1 Microsoft Defender XDR deployed and generating incidents . . . 7

2.2 The Microsoft Defender XDR data connector available in Sentinel . . . 7

2.3 Permission to configure the connector and incident sync . . . 8

2.4 Existing Microsoft incident-creation rules identified (de-duplication plan) . . . 8

2.5 A severity / priority alignment scheme . . . 8

2.6 SOC roles and an analyst group defined . . . 8

2.7 A pilot scope and success criteria . . . 9

Checklist . . . 9

Global Requirements — Sentinel Solutions 9

G1. An Azure subscription . . . 9

G2. A Log Analytics workspace . . . 10

G3. Microsoft Sentinel enabled on the workspace . . . 10

G4. Administrative roles (least privilege) . . . 10

G5. Ingestion and retention cost plan . . . 10

G6. Diagnostic settings capability on each source . . . 11

G7. Change control & rollback posture . . . 11

3. Implementation Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 11

Step 1 — Confirm the workspace and Sentinel . . . 12

Step 2 — Install and open the Microsoft Defender XDR connector . . . 12
Step 3 — Connect Defender XDR and enable incident synchronization . . . 12
Step 4 — Remove duplicate incidents (de-duplication) . . . 12
Step 5 — Align severity, priority, and ownership (automation rule) . . . 13
Step 6 — Scope RBAC for the SOC . . . 13
Step 7 — Adopt the triage and shift hand-off runbook . . . 13
Step 8 — Validate the pilot (the gate) . . . 14
Step 9 — Expand to the full SOC . . . 14
5. Testing Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 15
Test 1 — Connector connected and syncing . . . 15
Test 2 — Defender incidents appear in Sentinel . . . 15
Test 3 — No duplicate incidents (de-duplication holds) . . . 15
Test 4 — Severity / ownership / tags applied on arrival . . . 15
Test 5 — Bi-directional status sync . . . 15
Test 6 — Single-incident end-to-end (the correlation drill) . . . 15
Test 7 — Triage and hand-off runbook . . . 15
Test 8 — RBAC scope correct . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 16
A. Duplicate incidents in the queue . . . 16
B. Defender incidents not appearing in Sentinel . . . 17
C. Status not syncing between portals . . . 17
D. Severity / ownership not applied . . . 17
E. Permission errors . . . 17
F. Diagnostics quick reference (KQL in Microsoft Sentinel → Logs) . . . 18
G. When to escalate . . . 18
7. Best Practices — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 18
Where to go next . . . 19
8. Deliverables Summary — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents 19
Documentation . . . 19
PowerShell scripts . . . 19
What the buyer receives . . . 20
Outcome . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

Sentinel Solutions

Most security teams run Microsoft Sentinel and Microsoft Defender XDR as two separate consoles. The result is predictable: the same attack shows up as an incident in both places, analysts hop between portals, related identity, endpoint, email, and cloud signals never get stitched together, and two different severity schemes make it impossible to agree on what to work first. This solution connects Defender XDR to Sentinel, turns on incident synchronization, removes the duplicate incidents, and gives the SOC a single, correlated, prioritized queue with one triage workflow and a clean shift hand-off.

This is a complete, deploy-it-today package for onboarding a SOC onto a unified Sentinel + Defender XDR incident queue, written for an administrator with no prior SIEM experience.

What you get

- A plain-language Solution Overview of how Defender XDR and Sentinel connect, how incident sync and de-duplication work, and how correlation stitches identity, endpoint, email, and cloud signals into one incident.
- A Requirements Guide with verify / implement-if-missing for every solution-specific prerequisite.
- An Implementation Guide of precise, click-by-click portal steps in the required order — connect Defender XDR, enable incident sync, suppress duplicate incident creation, align severity/priority/ownership, and scope RBAC.
- An analyst **triage workflow and shift hand-off runbook** the SOC can adopt on day one.
- Five PowerShell scripts for readiness discovery, connector/sync validation, a duplicate-and-coverage report, an incident-queue health report, and a workspace/RBAC scoping helper.
- Testing and Troubleshooting guides for a confident rollout.

What this delivers

A unified incident queue: the Microsoft Defender XDR connector connected to `Security Telemetry - SOC - Workspace`, incident synchronization on so Defender XDR incidents and their status flow both ways, the separate Microsoft incident-creation rules turned off so no duplicate incidents are raised, an automation rule aligning severity/priority/ownership on arrival, and RBAC scoped so analysts see one prioritized list — validated on a pilot before the whole SOC is switched over.

The standard deployment model

`Assess -> Configure -> Pilot -> Validate -> Expand`, with a pilot gate that confirms sync works and duplicates are gone before the team moves off the old two-console workflow.

Boundary

This solution unifies and operationalizes the incident queue. Incident-response *determinations* — how to classify, contain, and remediate a given incident, and what is a true or false positive — remain the SOC's decision; the guide provides the technical wiring and the triage/hand-off process, not the response verdicts.

Outcome

One correlated, prioritized incident queue across identity, endpoint, email, and cloud — no duplicates, one severity scheme, clear ownership, and a repeatable triage and hand-off routine — proven on a pilot before full SOC cut-over.

1. Solution Overview — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

What this solution is

Microsoft Defender XDR (the Microsoft 365 Defender family: Defender for Endpoint, Defender for Office 365, Defender for Identity, and Defender for Cloud Apps) detects and correlates threats across endpoint, email, identity, and SaaS, and raises its own **incidents** in the Defender portal. Microsoft Sentinel is Microsoft's cloud-native SIEM/SOAR, running on a Log Analytics workspace, which raises incidents from its own analytics rules across every connected source. Run separately, the two produce overlapping and duplicated incidents.

This solution connects the two: the **Microsoft Defender XDR data connector** streams Defender XDR incidents and alerts into Sentinel and keeps their status synchronized, the duplicate-creating rules are turned off, and the SOC works one queue. Optionally, Sentinel can be onboarded into the unified security operations experience in the Defender portal so both live behind one pane of glass. *Verify current availability of the unified Defender + Sentinel experience in your tenant.*

How the connection works

1. Defender XDR generates incidents and alerts from its workloads (endpoint, email, identity, cloud apps), each already correlating the related alerts into a single incident.

2. The **Microsoft Defender XDR data connector** in Sentinel, with **incident synchronization** enabled, streams those incidents into Sentinel as Sentinel incidents and streams the underlying alerts into the workspace tables (for example `SecurityAlert`, `SecurityIncident`).

3. **Bi-directional sync** keeps the incident's status, owner, and classification aligned between the Defender portal and Sentinel — close it in one, it closes in the other.

4. To avoid duplicates, the separate **Microsoft incident-creation rules** (the older per-product rules that make a *second* Sentinel incident from each Defender alert) are turned **off**, so the connector's synced incident is the single source of truth.

5. Sentinel **automation rules** run on incident creation to align severity/priority, assign an owner or team, and tag the incident — so every incident lands in the queue already prioritized and owned.

6. Sentinel's built-in **Fusion** correlation and **alert-grouping** can further stitch related alerts (for example a risky sign-in followed by endpoint malware) into one multi-stage incident across sources.

The objects this solution creates (named to convention)

Every created object is named `<Purpose> - <Scope> - <Stage>` and reused verbatim throughout the guide, testing, and scripts:

- Workspace (destination): `Security Telemetry - SOC - Workspace`
- Defender XDR connector instance: `Microsoft Defender XDR - SOC - Pilot`
- Automation rule (severity/owner/tag alignment): `Incident Alignment - SOC - Pilot`
- Analyst assignment group / RBAC scope: `Microsoft Sentinel Responder - SOC - Pilot`

What you can control

- Whether Defender XDR incidents and/or the raw alerts stream into Sentinel, and whether sync is bi-directional.
- Which Microsoft incident-creation rules stay off (de-duplication) versus any you deliberately keep.
- The severity/priority mapping and the ownership/tagging applied on arrival (automation rule).
- The triage workflow and shift hand-off routine the analysts follow (the runbook).
- The RBAC scope: who can read, triage, and respond in the workspace.

Why this matters (the business case)

- **One queue** — analysts work a single prioritized list instead of reconciling two consoles.
- **No duplicates** — turning off the redundant incident-creation rules ends the "same incident twice" noise that wastes triage time.
- **Correlation** — identity, endpoint, email, and cloud signals are stitched into one incident, so multi-stage attacks are seen whole instead of as scattered fragments.
- **Aligned priority and ownership** — one severity scheme and automatic assignment mean the team agrees on what to work first and who owns it.
- **Clean hand-offs** — a documented triage and shift hand-off runbook keeps nothing dropped between shifts.

Limitations (be honest with buyers)

- The Defender XDR connector requires the relevant Defender workloads to be licensed and generating incidents; it does not create detections that your Defender licensing does not produce.
- Sync behavior and the unified Defender + Sentinel portal experience evolve; *verify current availability and behavior in your tenant.*
- This solution unifies and operationalizes the queue; it is not a detection-engineering or custom-analytics program (those build on this foundation).

• Incident-response verdicts (true/false positive, containment, remediation) remain the SOC's decision — the guide provides the wiring and workflow, not the response determination.

The standard deployment model

> *Confirm the workspace and Sentinel, connect Defender XDR and enable incident sync, suppress the duplicate incident-creation rules, add the severity/ownership automation rule, adopt the triage/hand-off runbook, validate on a pilot that sync works and duplicates are gone, then expand to the full SOC. Standard pattern:* `Assess -> Configure -> Pilot -> Validate -> Expand`.

2. Requirements Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

Verify **every** requirement before configuring. Each entry: **Description / Why / Verify / Implement-if-missing.** Category-wide prerequisites (Azure subscription, Log Analytics workspace, Microsoft Sentinel enabled, RBAC baseline, ingestion/retention cost plan, diagnostic-settings capability, change control) are in `global-requirements-sentinel-solutions.md`; do not repeat them — confirm them there first. For this solution the workspace is named `Security Telemetry - SOC - Workspace`.

2.1 Microsoft Defender XDR deployed and generating incidents

• **Description:** At least one Microsoft Defender XDR workload (Defender for Endpoint, Office 365, Identity, or Cloud Apps) onboarded and producing incidents in the Defender portal (`https://security.microsoft.com`).

• **Why:** The connector streams Defender XDR incidents into Sentinel; if Defender XDR raises no incidents there is nothing to unify.

• **Verify:** `security.microsoft.com` → `Incidents & alerts` → `Incidents` shows recent incidents. Run `SOCXDR-Discovery.ps1` to confirm the connected workloads.

• **Implement:** Onboard the relevant Defender workloads (licensing per Microsoft 365 E5 / E5 Security or the standalone plans — *verify current licensing in your tenant*) and confirm incidents are being raised.

2.2 The Microsoft Defender XDR data connector available in Sentinel

• **Description:** The Microsoft Defender XDR connector (and its solution package) present in the Sentinel Content hub for `Security Telemetry - SOC - Workspace`.

• **Why:** This connector is the supported way to bring Defender XDR incidents/alerts into Sentinel and enable synchronization.

• **Verify:** `portal.azure.com` → `Microsoft Sentinel` → (workspace) → `Configuration` → `Data connectors` → search `Microsoft Defender XDR`. Run `SOCXDR-Validate.ps1`.

• **Implement:** Install the **Microsoft Defender XDR** solution from `Content hub` if the connector is not listed.

2.3 Permission to configure the connector and incident sync

• **Description:** A role that can configure the Defender XDR connector in Sentinel and read incidents in the Defender portal — typically Microsoft Sentinel Contributor on the workspace plus a Defender role (for example Security Reader/Administrator) with global (tenant-level) Sentinel rights required by this connector.

• **Why:** Enabling incident sync writes connector configuration in Sentinel and reads from Defender XDR; both scopes are needed.

• **Verify:** On the workspace, `Access control (IAM)` shows Microsoft Sentinel Contributor; in `security.microsoft.com` → `Permissions` the account holds a Defender role. Run `SOCXDR-Discovery.ps1`.

• **Implement:** Assign Microsoft Sentinel Contributor at the workspace (or resource-group) scope and the least-privilege Defender role needed; prefer eligible (PIM) activation. *The Defender XDR connector requires tenant-scoped Sentinel permissions — verify the exact role in your tenant.*

2.4 Existing Microsoft incident-creation rules identified (de-duplication plan)

• **Description:** A list of any active **Microsoft incident-creation rules** in Sentinel that create incidents from the same Defender products (for example "Create incidents based on Microsoft Defender for Endpoint alerts").

• **Why:** If these stay on while the Defender XDR connector also creates synced incidents, every incident appears twice; the de-duplication step turns them off.

• **Verify:** `Microsoft Sentinel` → `Analytics` → `Active rules` → filter by rule type `Microsoft incident creation rule`. Run `SOCXDR-Report.ps1` to list them.

• **Implement:** Record which are active so Step 4 of the Implementation Guide can disable them cleanly (and so rollback is possible).

2.5 A severity / priority alignment scheme

• **Description:** An agreed mapping of how Defender XDR severities (High/Medium/Low/Informational) map to the SOC's working priority, and who owns each severity band.

• **Why:** A single, agreed scheme is what lets analysts work one prioritized list; the automation rule encodes it.

• **Verify:** A written severity→priority→owner map exists (even a one-page table).

• **Implement:** Agree the mapping with SOC leadership before building the automation rule `Incident Alignment - SOC - Pilot`.

2.6 SOC roles and an analyst group defined

• **Description:** An Entra ID security group for the SOC analysts/responders (for example `SOC-Analysts`) and a definition of who is Tier 1 (triage), Tier 2 (investigate), and lead.

• **Why:** RBAC is assigned to a group, and the triage/hand-off runbook assigns work by tier; both need the roster defined.

• **Verify:** `entra.microsoft.com` → `Groups` → the SOC group exists with the analysts as members.

• **Implement:** Create the group and populate it; map members to tiers for the runbook.

2.7 A pilot scope and success criteria

• **Description:** A named pilot — typically one or two Defender workloads and a small analyst cohort — with pass criteria: incidents sync, no duplicates appear, and severity/ownership is applied automatically.

• **Why:** The standard model validates the unified queue on a pilot before the whole SOC moves off the two-console workflow.

• **Verify:** The pilot workloads, cohort, and pass criteria are written down. Run `SOCXDR-Validate.ps1` to confirm sync during the pilot.

• **Implement:** Choose the minimum useful scope for the pilot; record the gate (sync confirmed, duplicates gone) before expanding.

Checklist

• Azure subscription, Log Analytics workspace, Microsoft Sentinel enabled, change control — see global requirements

• Defender XDR deployed and raising incidents — `SOCXDR-Discovery.ps1`

• Microsoft Defender XDR connector available in Content hub — `SOCXDR-Validate.ps1`

• Permission to configure the connector + incident sync

• Active Microsoft incident-creation rules listed — `SOCXDR-Report.ps1`

• Severity→priority→owner mapping agreed

• SOC analyst group and tier definitions in place

• Pilot scope and pass criteria (sync + no duplicates) defined

Global Requirements — Sentinel Solutions

These requirements are common to **every** Sentinel Solutions solution in the standard program model. They are the baseline Azure Monitor / Microsoft Sentinel platform prerequisites that any telemetry, detection, or SIEM engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named* `<Purpose> - <Scope> - <Stage>` *and reused verbatim throughout the engagement (for example* `Audit Telemetry - Tenant - Pilot`*).*

G1. An Azure subscription

• **Description:** An active Azure subscription in the tenant in scope, with billing enabled.

• **Why:** A Log Analytics workspace and Microsoft Sentinel are Azure resources billed against a subscription; nothing can be provisioned without one.

• **Verify:** Sign in to `https://portal.azure.com`, open `Subscriptions`, and confirm an active subscription resolves and shows status `Active`.

• **Implement-if-missing:** Create or associate an Azure subscription (Enterprise Agreement, CSP, or pay-as-you-go) and confirm billing is in place.

G2. A Log Analytics workspace

• **Description:** A Log Analytics workspace to receive and store ingested telemetry, sized to the data volume and retention in scope.

• **Why:** Microsoft Sentinel runs on top of a Log Analytics workspace; all log data lands here and all queries/analytics run against it.

• **Why region matters:** the workspace region determines data residency and affects egress; pick one deliberately before ingestion starts (it cannot be moved later).

• **Verify:** `portal.azure.com` → `Log Analytics workspaces` → confirm the target workspace exists, note its region and resource group.

• **Implement-if-missing:** Create a workspace named `Security Telemetry - Tenant - Workspace`; choose the region for residency; select the pricing tier that fits expected daily ingestion.

G3. Microsoft Sentinel enabled on the workspace

• **Description:** Microsoft Sentinel added to (enabled on) the Log Analytics workspace in scope.

• **Why:** Connectors, analytics rules, hunting, and incidents are Sentinel features layered on the workspace; without Sentinel enabled you have logs but no SIEM.

• **Verify:** `portal.azure.com` → `Microsoft Sentinel` → confirm the workspace is listed.

• **Implement-if-missing:** `Microsoft Sentinel` → `Create` → select the workspace → `Add`.

G4. Administrative roles (least privilege)

• **Description:** Appropriately scoped roles for the work — Microsoft Sentinel Contributor (or Log Analytics Contributor) on the workspace for configuration, plus the tenant roles needed to set diagnostic settings on the sources (for example a role that can edit Entra ID diagnostic settings).

• **Why:** Each configuration action requires sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** On the workspace/resource group, `Access control (IAM)` → confirm the operating accounts hold the required roles. In Entra → `Roles & admins`, confirm the role that can edit diagnostic settings.

• **Implement-if-missing:** Assign the minimum roles needed at the resource-group or workspace scope; prefer eligible (PIM) activation over standing access where available.

G5. Ingestion and retention cost plan

• **Description:** A documented estimate of daily ingestion (GB/day) and a retention decision (interactive retention vs long-term/archive), with a budget owner.

• **Why:** Sentinel and Log Analytics are billed primarily on data ingested and data retained; identity and device audit logs can be high-volume, so cost must be planned before connectors are switched on.

• **Verify:** Confirm a written estimate exists (sources × expected GB/day) and that retention periods are agreed per table or workspace.

• **Implement-if-missing:** Estimate volume from the source log categories in scope; choose pay-as-you-go or a commitment tier; set workspace/table retention; record the budget and owner.

G6. Diagnostic settings capability on each source

• **Description:** The ability to create diagnostic settings (or enable the relevant data connector) on every source whose telemetry is in scope.

• **Why:** Telemetry only flows to the workspace when a diagnostic setting (or connector) routes the source's log categories to it; this is the switch that turns the pipeline on.

• **Verify:** For each source, confirm a diagnostic setting can be created and that no existing setting already routes the same categories elsewhere (to avoid duplicate ingestion and double billing).

• **Implement-if-missing:** Plan one named diagnostic setting per source that forwards the in-scope categories to the workspace (named per G's convention).

G7. Change control & rollback posture

• **Description:** An approved change process with a scheduled window, plus a documented rollback (which diagnostic settings/connectors/analytics rules were added) before any change.

• **Why:** Enabling connectors and analytics rules changes ingestion volume (and therefore cost) and can generate incidents; controlled changes and a recorded back-out keep the pipeline safe and auditable.

• **Verify:** Confirm an approved change record and a written list of objects to be created (so they can be removed cleanly).

• **Implement-if-missing:** Stand up a lightweight change/approval process; capture the pre-change configuration; document the rollback (delete the named diagnostic setting / disable the connector / disable the rule).

3. Implementation Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

Follow these steps **in order**. They use two portals: the Azure portal (`https://portal.azure.com`) for Microsoft Sentinel, the connector, analytics, automation, and RBAC; and the Microsoft Defender portal (`https://security.microsoft.com`) for Defender XDR incidents and the optional unified experience.

> *Model: connect Defender XDR, turn on incident sync, remove duplicate incident creation, align severity/ownership, adopt the triage/hand-off runbook, validate on a pilot, then expand. Every object created is named* `<Purpose> - <Scope> - <Stage>` *and reused verbatim.*

Objects this guide creates (name them exactly):

- Workspace (destination): `Security Telemetry - SOC - Workspace`
- Defender XDR connector instance: `Microsoft Defender XDR - SOC - Pilot`
- Automation rule: `Incident Alignment - SOC - Pilot`
- RBAC scope / assignment: `Microsoft Sentinel Responder - SOC - Pilot`

Step 1 — Confirm the workspace and Sentinel

1. `portal.azure.com` → search `Log Analytics workspaces` → confirm `Security Telemetry - SOC - Workspace` exists (create it if not; see global requirements G2). Note its region and resource group.

2. `portal.azure.com` → search `Microsoft Sentinel` → confirm the workspace is listed. If not: `Create` → select the workspace → `Add`.

Step 2 — Install and open the Microsoft Defender XDR connector

1. `Microsoft Sentinel` → select `Security Telemetry - SOC - Workspace` → `Content management` → `Content hub`.

2. Search `Microsoft Defender XDR` → select the solution → `Install` (skip if already installed).

3. `Configuration` → `Data connectors` → search `Microsoft Defender XDR` → `Open connector page`.

Step 3 — Connect Defender XDR and enable incident synchronization

1. On the Microsoft Defender XDR connector page, click `Connect incidents & alerts`. This creates the connector instance; record it as `Microsoft Defender XDR - SOC - Pilot`.

2. Leave `Turn off all Microsoft incident creation rules for these products` available for Step 4 — the connector page offers this toggle; you can use it here to disable the duplicate rules in one click, or do it manually in Step 4.

3. Under the connected data types, select the Defender workloads in your pilot scope (for example Defender for Endpoint and Defender for Office 365) so their alerts also stream into the workspace tables.

4. Click `Apply Changes` / `Connect`. Incident sync begins; new Defender XDR incidents now appear in Sentinel, and status/owner/classification sync both ways.

Step 4 — Remove duplicate incidents (de-duplication)

1. `Microsoft Sentinel` → `Configuration` → `Analytics` → `Active rules`.

2. Filter `Rule type` = `Microsoft incident creation rule`. These are the older rules that make a *second* Sentinel incident from each Defender alert.

3. For every such rule covering a product now synced by the Defender XDR connector, select it → `Disable` (do not delete — disabling keeps rollback trivial). Use the list captured in Requirement 2.4 /

`SOCXDR-Report.ps1` so none are missed.

4. Result: each threat now produces exactly one synced incident from the connector, not a duplicate pair.

Step 5 — Align severity, priority, and ownership (automation rule)

1. `Microsoft Sentinel` → `Configuration` → `Automation` → `Create` → `Automation rule`.

2. Name: `Incident Alignment - SOC - Pilot`.

3. `Trigger`: `When incident is created`.

4. `Conditions`: scope to the pilot — for example `Alert product names Contains` the Defender products in scope (leave broad for a full-SOC rollout later).

5. `Actions`, added in order to encode your Requirement 2.5 scheme:

• `Change severity` — only if your scheme overrides the incoming severity for a band (otherwise omit and keep Defender's severity).

• `Assign owner` → the Tier 1 triage queue owner or the `SOC-Analysts` group, so every incident lands owned.

• `Add tags` → for example `SOC-Triage`, plus a priority tag such as `P1`/`P2` derived from severity, so the queue can be sorted and filtered by one scheme.

1. `Order`: `1`. `Apply`. New incidents now arrive already prioritized, owned, and tagged.

Step 6 — Scope RBAC for the SOC

1. `portal.azure.com` → `Log Analytics workspaces` → `Security Telemetry - SOC - Workspace` → `Access control (IAM)` → `Add role assignment`.

2. Assign to the `SOC-Analysts` group:

• `Microsoft Sentinel Responder` — triage/investigate/respond on incidents (Tier 1/Tier 2). Record as `Microsoft Sentinel Responder - SOC - Pilot`.

• `Microsoft Sentinel Contributor` — for SOC leads who also manage rules/automation.

1. Keep read-only stakeholders on `Microsoft Sentinel Reader`. Prefer eligible (PIM) activation for the higher roles.

2. `SOCXDR-Set-SocRbacScope.ps1` reports current assignments and can apply the scoped Responder role.

Step 7 — Adopt the triage and shift hand-off runbook

Give analysts one queue and one routine (this is the operational core — see the Testing Guide for the drill that proves it):

1. **The single queue.** `Microsoft Sentinel` → `Threat management` → `Incidents`. Sort by the priority tag from Step 5 (or by severity). This — not the two separate consoles — is now the SOC's working list.

2. **Tier 1 triage (per incident, top of queue first):** open the incident → review the correlated alerts, entities (user, device, IP, mailbox), and the incident graph → set `Status` = `Active`, confirm/adjust `Owner`, and either resolve (with classification) or escalate to Tier 2 by re-assigning owner and adding the `Tier2` tag.

3. **Tier 2 investigation:** work the escalated incidents; use the entity pages and, where present, the unified Defender investigation. Record findings in the incident `Comments`.

4. **Shift hand-off (end of every shift):** filter the queue to `Status != Closed` and `Owner = <this shift>`; for each open incident add a hand-off comment (state, next action, blockers); re-assign owner to the incoming shift group; confirm no `New`/untriaged incident is older than the agreed SLA. `SOCXDR-Report.ps1` prints the open-incident hand-off list.

5. **Bi-directional close:** closing the incident in Sentinel closes it in the Defender portal (and vice versa) — so analysts never need to reconcile the two.

Step 8 — Validate the pilot (the gate)

1. Run `SOCXDR-Validate.ps1` → confirms the Defender XDR connector is connected and incidents/alerts are arriving.

2. Run `SOCXDR-Report.ps1` → confirms the Microsoft incident-creation rules are disabled (no duplicate source), lists open incidents, and shows the hand-off view.

3. Trigger a benign Defender test detection (for example the Defender for Endpoint EICAR/test alert per Microsoft's guidance) and confirm it appears **once** in the Sentinel queue, already owned and tagged.

4. Pilot gate: do not cut the whole SOC over until sync is confirmed, no duplicate incident appears for the test, and the automation rule applied severity/owner/tag.

Step 9 — Expand to the full SOC

1. Once the gate passes, broaden the automation rule scope (or clone it to `Incident Alignment - SOC - Production`) to cover all Defender products and Sentinel analytics incidents.

2. Add the remaining Defender workloads to the connector's streamed data types; disable any remaining Microsoft incident-creation rules for them.

3. Onboard the whole analyst roster to `Microsoft Sentinel Responder`; retire the old two-console triage habit.

4. Optionally onboard Sentinel into the **unified security operations experience** in the Defender portal so both live behind one pane of glass. *Verify current availability in your tenant.*

5. Re-run `SOCXDR-Report.ps1` each cycle to confirm no duplicate rules creep back and the hand-off list stays clean.

Order of operations

```
Workspace + Sentinel -> install & connect Defender XDR (incident sync) ->
disable Microsoft incident-creation rules (de-dup) -> severity/owner
automation rule -> RBAC -> triage & hand-off runbook -> validate pilot (gate)
-> expand
```

5. Testing Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

Run after the pilot is configured (Implementation Step 8) and after each expansion. Scripts referenced: `SOCXDR-Discovery.ps1`, `SOCXDR-Validate.ps1`, `SOCXDR-Report.ps1`, `SOCXDR-IncidentQueue.ps1`, `SOCXDR-Set-SocRbacScope.ps1`.

Test 1 — Connector connected and syncing

Do: `portal.azure.com` → `Microsoft Sentinel` → `Security Telemetry - SOC - Workspace` → `Data connectors` → `Microsoft Defender XDR`, or run `SOCXDR-Validate.ps1`. **Expected:** Connector status is `Connected` (`Microsoft Defender XDR - SOC - Pilot`); the data-received indicator is recent; incidents & alerts sync is on.

Test 2 — Defender incidents appear in Sentinel

Do: `Microsoft Sentinel` → `Threat management` → `Incidents`; or `Logs` → `SecurityIncident | where TimeGenerated > ago(24h) | take 10`. **Expected:** Recent Defender XDR incidents are present in the Sentinel queue with their source workload shown.

Test 3 — No duplicate incidents (de-duplication holds)

Do: `Microsoft Sentinel` → `Analytics` → `Active rules` → filter `Microsoft incident creation rule`; or run `SOCXDR-Report.ps1`. **Expected:** The Microsoft incident-creation rules for the synced products are `Disabled`. A single test detection (Test 6) produces **one** incident, not a pair.

Test 4 — Severity / ownership / tags applied on arrival

Do: Open the newest incident in the queue, or run `SOCXDR-IncidentQueue.ps1`. **Expected:** The incident already has an owner (or SOC group) and the priority/`SOC-Triage` tags from `Incident Alignment - SOC - Pilot`; severity matches the agreed scheme.

Test 5 — Bi-directional status sync

Do: In Sentinel, set a test incident's `Status` to `Active`, then `Closed` with a classification. Open the same incident in `security.microsoft.com` → `Incidents`. **Expected:** The status/owner/classification change is reflected in the Defender portal (and a change made in Defender reflects back in Sentinel).

Test 6 — Single-incident end-to-end (the correlation drill)

Do: Generate a benign Defender test detection (for example the Defender for Endpoint test/EICAR alert per Microsoft's guidance) on a pilot device. **Expected:** Exactly one Sentinel incident is created, correlating the related alert(s) and entities (device/user), already owned and tagged — no duplicate from a Microsoft incident-creation rule.

Test 7 — Triage and hand-off runbook

Do: Walk one incident through the runbook (Implementation Step 7): triage → escalate → hand-off comment → re-assign to next shift. Run `SOCXDR-Report.ps1` for the open-incident hand-off list.
Expected: The incident carries the escalation tag, a hand-off comment, and the new owner; the hand-off list shows no untriaged incident older than the SLA.

Test 8 — RBAC scope correct

Do: Sign in as a `SOC-Analysts` member (or run `SOCXDR-Set-SocRbacScope.ps1` in report mode).
Expected: Analysts can triage/respond on incidents (`Microsoft Sentinel Responder`); read-only stakeholders cannot modify; leads hold `Microsoft Sentinel Contributor`.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-6 pass: connector syncing, Defender incidents in the queue, duplicate rules disabled, severity/owner/tags applied, bi-directional sync works, and one test detection = one incident
Expand pass	Tests 2, 3, 4, 7 pass across all Defender products and the full analyst roster, with the hand-off list clean, before retiring the two-console workflow

6. Troubleshooting Guide — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

Use `SOCXDR-Discovery.ps1` for readiness, `SOCXDR-Validate.ps1` for connector/sync state, `SOCXDR-Report.ps1` for duplicate-rule and hand-off status, `SOCXDR-IncidentQueue.ps1` for queue health, and `SOCXDR-Set-SocRbacScope.ps1` for RBAC.

A. Duplicate incidents in the queue

Symptom	Cause	Remediation
Every threat appears as two incidents	A Microsoft incident-creation rule is still active alongside the Defender XDR connector	`Analytics` → `Active rules` → filter `Microsoft incident creation rule` → `Disable` the rules for the synced products (Implementation Step 4); run `SOCXDR-Report.ps1` to confirm none remain
Duplicates returned after a while	A disabled rule was re-enabled, or a new product was added without disabling its rule	Re-run `SOCXDR-Report.ps1` each cycle; disable the offending rule

B. Defender incidents not appearing in Sentinel

Symptom	Cause	Remediation
No Defender XDR incidents in Sentinel	Connector not connected, or incident sync not enabled	`Data connectors` → `Microsoft Defender XDR` → `Connect incidents & alerts`; run `SOCXDR-Validate.ps1`
Only alerts, no incidents	Alerts streamed but incident sync off	On the connector page enable incidents & alerts sync (not only the alert data types)
Sync delayed	First sync can lag	Wait, then re-run `SOCXDR-Validate.ps1`; confirm the connector `data received` timestamp is recent

C. Status not syncing between portals

Symptom	Cause	Remediation
Closing in Sentinel does not close in Defender	Bi-directional sync not established, or the incident predates sync	Confirm the connector's incident sync is on; test on a *new* incident created after sync was enabled (Test 5)
Owner/classification differs between portals	Change made during a sync delay	Allow sync to settle; avoid editing the same incident simultaneously in both portals

D. Severity / ownership not applied

Symptom	Cause	Remediation
New incidents arrive unowned / untagged	Automation rule missing, disabled, or its conditions don't match	Confirm `Incident Alignment - SOC - Pilot` exists, is enabled, order `1`, and its conditions match the products in scope (Implementation Step 5)
Wrong severity applied	`Change severity` action too broad	Narrow the automation rule conditions, or remove the severity override and keep Defender's severity

E. Permission errors

• Cannot configure the connector / enable sync → the account lacks the tenant-scoped Sentinel permission or the Defender role (Requirement 2.3).

• Analysts cannot triage incidents → assign `Microsoft Sentinel Responder` on the workspace (Implementation Step 6); use `SOCXDR-Set-SocRbacScope.ps1`.

F. Diagnostics quick reference (KQL in Microsoft Sentinel → Logs)

```
SecurityIncident | where TimeGenerated > ago(24h) | summarize count() by Severity, Status
SecurityAlert    | where TimeGenerated > ago(24h) | summarize count() by ProductName
SecurityIncident | where TimeGenerated > ago(24h) | where isempty(Owner) | project IncidentName, Severity,
```

G. When to escalate

If the connector shows `Connected`, incident sync is on, the Microsoft incident-creation rules are disabled, and incidents still duplicate or fail to sync after a full sync interval and a re-validation, capture the output of `SOCXDR-Validate.ps1` and `SOCXDR-Report.ps1` and open a Microsoft support case for the Defender XDR connector / unified experience on the workspace.

7. Best Practices — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

• **Pilot the unified queue before cut-over.** Connect one or two Defender workloads and a small cohort, confirm sync works and duplicates are gone, then move the whole SOC — never switch everyone at once.

• **Turn off the Microsoft incident-creation rules, don't delete them.** Disabling de-duplicates the queue while keeping a one-click rollback; deleting throws away that safety net.

• **Let the connector's synced incident be the single source of truth.** With incident sync on, work incidents in Sentinel (or the unified Defender portal) — not in two places — and rely on bi-directional sync to keep status aligned.

• **Keep Defender's severity unless you have a reason to override.** Defender XDR severity is already correlated across workloads; only remap it in the automation rule where your SOC's priority scheme genuinely differs, and encode that scheme once.

• **Assign an owner on arrival.** An automation rule that assigns owner and priority tag on creation is what turns a pile of incidents into a workable, sortable queue — no incident should sit unowned.

• **Make the hand-off a ritual, not an afterthought.** End every shift by clearing untriaged incidents, adding a hand-off comment per open incident, and re-assigning to the next shift; run the report so nothing older than the SLA is dropped.

• **Name every object to the convention.** Use `<Purpose> - <Scope> - <Stage>` for the workspace, connector instance, automation rule, and RBAC scope, and reuse the names verbatim — it makes the SOC's configuration auditable and repeatable.

• **Right-size RBAC.** Give analysts `Microsoft Sentinel Responder`, leads `Microsoft Sentinel Contributor`, and stakeholders `Microsoft Sentinel Reader`; prefer eligible (PIM) activation over standing access.

• **Lean on correlation.** Let Sentinel's Fusion and alert-grouping stitch related alerts into multi-stage incidents rather than triaging each alert alone — it is the point of unifying the queue.

• **Keep response decisions with the SOC.** The pipeline delivers a clean, correlated, prioritized queue; classification, containment, and remediation verdicts remain the analysts' call, recorded in the incident.

Where to go next

• **Detection engineering / custom analytics** — author Sentinel analytics rules and hunting queries on top of the unified queue.

• **SOAR / automation playbooks** — extend automation rules with Logic App playbooks for enrichment and response actions (pilot/simulate before enforcing any response).

• **Audit telemetry onboarding** — feed Entra ID and Intune audit logs into the same workspace so identity and device signal enriches correlation.

• **Threat intelligence and hunting** — add TI feeds and proactive hunting to the same Sentinel foundation.

8. Deliverables Summary — SOC Onboarding: Microsoft Sentinel + Defender XDR Unified Incidents

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	How Defender XDR connects to Sentinel, incident sync and de-duplication, correlation, the objects created, benefits, limitations
2. Requirements Guide	Every solution-specific prerequisite with Verify / Implement-if-missing
Global Requirements (Sentinel Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal steps (connect Defender XDR, incident sync, disable duplicate rules, severity/owner automation, RBAC, triage & hand-off runbook)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations and where to go next

PowerShell scripts

Script	What it does
`SOCXDR-Discovery.ps1`	Reports readiness — workspace, Sentinel enabled, the Microsoft Defender XDR connector state, and connected Defender workloads (read-only)

Script	What it does
`SOCXDR-Validate.ps1`	Confirms the Defender XDR connector is connected and incidents/alerts are arriving in the workspace (read-only)
`SOCXDR-Report.ps1`	Lists active Microsoft incident-creation rules (duplicate risk), disabled state, and the open-incident shift hand-off view (read-only)
`SOCXDR-IncidentQueue.ps1`	Incident-queue health: open incidents by severity/owner/tag, unowned/untriaged and aging incidents against SLA (read-only)
`SOCXDR-Set-SocRbacScope.ps1`	Workspace/RBAC scoping helper — reports current assignments and can apply the scoped Microsoft Sentinel Responder role (default preview / `-WhatIf`)

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process following the standard model: `Assess -> Configure -> Pilot -> Validate -> Expand`.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

One correlated, prioritized incident queue across identity, endpoint, email, and cloud — Defender XDR connected to Sentinel, incident sync on, duplicates removed, severity/ownership aligned, and a triage/hand-off runbook the SOC runs every shift — validated on a pilot before full cut-over.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOCXDR-Discovery.ps1

Purpose: Reports whether the building blocks are in place: the Log Analytics workspace, Microsoft Sentinel on it, the Microsoft Defender XDR data connector and its state, and whether any Microsoft incident-creation rules exist (duplicate-incident risk). Read-only — makes no changes. HOW TO RUN Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-Discovery.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights. EXPECTED RESULTS Workspace found, Sentinel enabled (yes/no), the Defender XDR connector state, and a count of Microsoft incident-creation rules to disable. Highlights anything missing. .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName)

How to run: Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-Discovery.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights. EXPECTED RESULTS Workspace found, Sentinel enabled (yes/no), the Defender XDR connector state, and a count of Microsoft incident-creation rules to disable. Highlights anything missing. .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName)

SOCXDR-IncidentQueue.ps1

Purpose: Gives SOC leadership a one-look health view of the unified queue: open incidents grouped by severity and by owner, the count that are UNOWNED or untriaged, whether the alignment tags (e.g. SOC-Triage / priority) are present, and the oldest open incident. Read-only. HOW TO RUN Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-IncidentQueue.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" Requires modules: Az.Accounts, Az.SecurityInsights. EXPECTED RESULTS Counts by severity and status, an owned-vs-unowned split, a tag-coverage figure (how many incidents carry the alignment tags), and the oldest open incident age. .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [string]$AlignmentTag = 'SOC-Triage')

How to run: Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-IncidentQueue.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" Requires modules: Az.Accounts, Az.SecurityInsights. EXPECTED RESULTS Counts by severity and status, an owned-vs-unowned split, a tag-coverage figure (how many incidents carry the alignment tags), and the oldest open incident age. .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [string]$AlignmentTag = 'SOC-Triage')

SOCXDR-Report.ps1

Purpose: Two reports the SOC needs every cycle: (1) which Microsoft incident-creation rules are still ENABLED (duplicate-incident risk that should be disabled), and (2) the open-incident hand-off view — incidents not yet closed, with owner, severity, age, and whether they breach the SLA. Read-only — makes no changes. HOW TO RUN Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-Report.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" [-SlaHours 4] Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights. EXPECTED RESULTS A list of enabled Microsoft incident-creation rules (should be empty after de-duplication) and a table of open incidents for shift hand-off, flagging any older than -SlaHours. .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [int]$SlaHours = 4)

How to run: Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-Report.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" [-SlaHours 4] Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights. EXPECTED RESULTS A list of enabled Microsoft incident-creation rules (should be empty after de-duplication) and a table of open incidents for shift hand-off, flagging any older than -SlaHours. .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [int]$SlaHours = 4)

SOCXDR-Set-SocRbacScope.ps1

Purpose: Workspace/RBAC scoping helper for the SOC. Reports current role assignments on the workspace, and can assign 'Microsoft Sentinel Responder' to the SOC analyst group at the workspace scope. Defaults to -WhatIf (reports what it WOULD do, changes nothing). HOW TO RUN Connect-AzAccount first. Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\SOCXDR-Set-SocRbacScope.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" -SocGroupObjectId <entra-group-object-id> Apply for real (must pass -WhatIf:$false): powershell -ExecutionPolicy Bypass -File .\SOCXDR-Set-SocRbacScope.ps1 -ResourceGroup <rg> -WorkspaceName "..." -SocGroupObjectId <id> -WhatIf:$false Requires modules: Az.Accounts, Az.Resources, Az.OperationalInsights. Requires Owner/User Access Administrator on the workspace scope to assign roles. EXPECTED RESULTS In preview: lists existing assignments and prints the assignment it WOULD create. When applied: assigns 'Microsoft Sentinel Responder' to the SOC group at the workspace scope (recorded as 'Microsoft Sentinel Responder - SOC - Pilot'). .NOTES Sentinel Solutions. White-label. Mutating only when -WhatIf:$false is passed. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [Parameter(Mandatory)] [string]$SocGroupObjectId, [string]$RoleName = 'Microsoft Sentinel Responder', [bool]$WhatIf = $true)

How to run: Connect-AzAccount first. Preview (default, safe): powershell -ExecutionPolicy Bypass -File .\SOCXDR-Set-SocRbacScope.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" -SocGroupObjectId <entra-group-object-id> Apply for real (must pass -WhatIf:$false): powershell -ExecutionPolicy Bypass -File .\SOCXDR-Set-SocRbacScope.ps1 -ResourceGroup <rg> -WorkspaceName "..." -SocGroupObjectId <id> -WhatIf:$false Requires modules: Az.Accounts, Az.Resources, Az.OperationalInsights. Requires Owner/User Access Administrator on the workspace scope to assign roles. EXPECTED RESULTS In preview: lists existing assignments and prints the assignment it WOULD create. When applied: assigns 'Microsoft Sentinel Responder' to the SOC group at the workspace scope (recorded as 'Microsoft Sentinel Responder - SOC - Pilot'). .NOTES Sentinel Solutions. White-label. Mutating only when -WhatIf:$false is passed. > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([Parameter(Mandatory)]

[string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [Parameter(Mandatory)] [string]$SocGroupObjectId, [string]$RoleName = 'Microsoft Sentinel Responder', [bool]$WhatIf = $true)

SOCXDR-Validate.ps1

Purpose: Validates the unified-queue pipeline: the Microsoft Defender XDR connector is present, and Defender incidents/alerts have actually landed in the workspace (SecurityIncident / SecurityAlert) within a recent window. Read-only — makes no changes. HOW TO RUN Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-Validate.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" [-LookbackHours 24] Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights, Az.OperationalInsights (query). EXPECTED RESULTS Connector present = yes/no; recent SecurityIncident and SecurityAlert row counts > 0 when sync is healthy. Flags an empty result as "no data arriving — check sync". .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [int]$LookbackHours = 24)

How to run: Connect-AzAccount first, then: powershell -ExecutionPolicy Bypass -File .\SOCXDR-Validate.ps1 -ResourceGroup <rg> -WorkspaceName "Security Telemetry - SOC - Workspace" [-LookbackHours 24] Requires modules: Az.Accounts, Az.OperationalInsights, Az.SecurityInsights, Az.OperationalInsights (query). EXPECTED RESULTS Connector present = yes/no; recent SecurityIncident and SecurityAlert row counts > 0 when sync is healthy. Flags an empty result as "no data arriving — check sync". .NOTES Sentinel Solutions. White-label. Read-only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ResourceGroup, [Parameter(Mandatory)] [string]$WorkspaceName, [int]$LookbackHours = 24)